empresa: el cluster con las PCs del aula

`franki678/bigdata-sandbox:empresa` convierte un laboratorio de PCs con Windows en un **cluster Hadoop de verdad**. La PC del profe es el **maestro**: tiene el NameNode, el ResourceManager, Hive, Spark History y la interfaz web con la vista **Aula**. Cada PC de alumno se suma como **trabajador**, con un DataNode (guarda bloques de HDFS) y un NodeManager (corre tareas de YARN). Las PCs se conectan por un túnel WireGuard que arma el propio contenedor, sin tocar la red de la facultad.

Todo el paquete del aula son cuatro archivos, que se bajan del release `aula-v2`:

- `bigdata-sandbox-empresa.tar.gz`: la imagen del maestro;
- `bigdata-sandbox-trabajador.tar.gz` (y su variante `-arm64`): la imagen liviana de las PCs, `:empresa-trabajador`;
- `empresa-profe.cmd`: el que corre el profe;
- `empresa-alumno.cmd`: el que se manda a las PCs por Veyon.

Archivo	Para qué
README.md / README.pdf	Este documento: qué es, cómo está hecha, qué hace y qué no
guia.md / guia.pdf	Paso a paso: de armar el servidor a dar la clase
interactivo.html	Mapa clickeable del aula: piezas, puertos y conexiones (se abre sin internet)

Qué es

Es la imagen full más lo necesario para repartir el cluster entre máquinas:

- la red del aula (WireGuard);
- la inscripción de PCs con un código de clase;
- los racks por fila del aula;
- una imagen de trabajador recortada;
- la vista **Aula**, para proyectar en clase: qué PC está viva, qué corre en cada una, por dónde pasa el tráfico y qué pasa cuando una se apaga.

El objetivo pedagógico es que el alumno *vea* el cluster. Si apaga su PC, sus bloques se re-copian en otras y sus tareas se reintentan en otra máquina. Si corre un join, el shuffle cruza la red del aula.

Cómo se desarrolló

Sale del mismo `Dockerfile` que el resto, con dos etapas nuevas:

```
full ──► empresa            (maestro; también sirve de trabajador "completo")
wireguard-go ─┘
ubuntu:24.04 ──► trabajador  (sólo DataNode + NodeManager; :empresa-trabajador)
hadoop-trabajador ─┘   (Hadoop recortado)
```

- **wireguard-go** se compila en Go puro (`golang:1.26-alpine`, `CGO_ENABLED=0`). Es el plan B para cuando el kernel de Docker Desktop no trae el módulo WireGuard.
- **empresa** (`FROM full`):
  - suma `wireguard-tools`, `iproute2`, `nftables` y `wireguard-go`;
  - copia `empresa/arranque.sh` como punto de entrada y `empresa/aula/` (red, inscripción, racks, bloques);
  - fija `SANDBOX_VERSION=empresa` y `TERMINAL=0`.
- **trabajador** parte de `ubuntu:24.04` con JRE Temurin 8 y un Hadoop recortado: sin documentación ni ejemplos, pasa de 355 MB a 101 MB. Trae Python 3.12 con el numpy de Ubuntu (1.26.4); al armar se verifica que sea el mismo que fija `conf/requirements-ml.txt`. Spark y Tez no van en la imagen: le llegan por HDFS (`spark.yarn.archive`, `tez.lib.uris`). Comprimida pesa unos 192 MB, contra 1,6 GB de empresa, según las mediciones del workflow.

La vista Aula en vivo se agregó sobre el servidor de full:

- `server/sandbox_server/aula.py` arma el estado del aula leyendo:
  - las PCs inscriptas;
  - el JMX del NameNode;
  - los nodos de YARN;
  - `wg show`;
  - el plano.
- `server/sandbox_server/aula_vivo.py` muestrea el tráfico y las caídas.
- `ui/src/pages/aula/` es la pantalla.

Las pruebas y el release están en `.github/workflows/empresa.yml`:

- **Cuándo corre:** con cada push a `main` que toca `empresa/**`, `pruebas/aula.sh` o `server/sandbox_server/aula*.py`, y en las ramas `empresa-*`.
- **Armado:** arma base → ml → empresa → trabajador. Mide el `.tar.gz` del trabajador, lo carga con `docker load` y revisa la sintaxis de los `.cmd`.
- **Pruebas** (`pruebas/aula.sh`):
  - aula simulada en la red de Docker;
  - caída de una PC;
  - túnel WireGuard;
  - caída con túnel;
  - borrar de cero;
  - humo de full;
  - regresión.
- **Publicación:** sólo a mano (`workflow_dispatch`, desde `main`). Sube `:empresa` y `:empresa-trabajador` para amd64 y arm64. Si se pide, también arma el release del aula (por defecto `aula-v2`). Nunca toca `latest`, `base`, `ml` ni `full`.

Corridas medidas de ese workflow: 2 h 10 min 43 s (run 40714239592), 57 min 18 s (run 40819945257) y 1 h 14 min 28 s (run 40427694172).

Herramientas y versiones

Pieza	Versión / detalle
Todo lo de full	Hadoop 3.3.6, Hive 4.0.0, Tez 0.10.3, Spark 3.5.9, etc. (ver full)
WireGuard	`wireguard-tools` de Ubuntu + módulo del kernel, o `wireguard-go` v0.0.0-20250521234502-f333402bd9cb como respaldo
Firewall / conteo	`nftables` (tabla `inet aula_trafico`, sólo cuenta, no bloquea)
Inscripción	`empresa/aula/inscripcion.py`, sólo biblioteca estándar de Python
Trabajador	Ubuntu 24.04, Temurin 8 JRE, Hadoop 3.3.6 recortado, Python 3.12, numpy 1.26.4, `tini`
Scripts de Windows	`.cmd` con PowerShell adentro (no piden administrador), Docker Desktop
Reparto de PCs	Veyon (Transferencia de archivos + Ejecutar programa), que ya usa el laboratorio

Qué hace

Los roles

La variable `ROL` decide qué es cada contenedor:

`ROL`	Qué corre
`solo`	Igual que full: todo en un contenedor
`maestro`	NameNode y Secondary, ResourceManager, JobHistory, MariaDB, Hive (Metastore + HiveServer2), Spark History, la interfaz con la vista Aula y un **DataNode de archivo**
`trabajador`	Un DataNode y un NodeManager

El DataNode del maestro es de tipo ARCHIVE con política WARM: se queda con **una copia de todo**. Por eso, cuando termina la clase, se pueden apagar las PCs sin perder datos. La vista Aula avisa cuándo: «Ya pueden apagar las PCs».

Por defecto el maestro no corre tareas: no tiene NodeManager, salvo `MAESTRO_NM=si`.

La red del aula (`RED=wireguard`)

Así se conectan las PCs al maestro:

1. **El hub del túnel:** el maestro levanta `wg0` en `10.13.0.1/24`, en el puerto UDP **51820**, y abre la **inscripción** en el TCP **8099**.
2. **La inscripción:** cada PC pide un nombre (`POST /reservar`) y se une (`POST /unirse`) con el **código de la clase**, que son 4 letras de un alfabeto sin caracteres confusos. La PC `pcNN` queda en `10.13.0.(10+NN)`. Entran hasta 240 PCs.
3. **El túnel en marcha:**
   - `/etc/hosts` recibe los nombres del aula;
   - el keepalive es de 25 s;
   - la MTU es 1380, con respaldo automático a 1280 si los paquetes grandes no pasan.

4. **El driver:** si el kernel tiene WireGuard usa el módulo; si no, usa `wireguard-go`, que necesita `/dev/net/tun`. En los dos casos hace falta `--cap-add NET_ADMIN`.
5. **Clase nueva:** cada clase tiene un id. Si una PC se une a un maestro nuevo, borra su HDFS viejo (`/hadoop/dfs`, `/hadoop/tmp`) para no mezclar clases.

Con `RED=docker` todo pasa por la red de Docker, por nombre. Es el **aula simulada** (`empresa/aula-simulada.yml`), para probar en una sola compu.

Racks = filas del aula

El script `topologia` le dice a Hadoop en qué rack está cada nodo:

Nodo	Rack
maestro	`/profe`
pcNN que está en el plano	su fila del plano
pcNN que no está en el plano	`/fila((N-1)/PCS_POR_FILA+1)`, con 5 PCs por fila de fábrica
cualquier otro	`/default-rack`

Así HDFS reparte las copias entre filas distintas, como en un datacenter. Desde la vista Aula se edita el **plano** (8 filas × 12 lugares) y se guarda en el maestro. Hadoop lo lee cuando cada DataNode se registra, no en caliente.

La vista Aula (tecla `a`, sólo en el maestro)

Está pensada para proyectar con la Demostración de Veyon, en pantalla completa:

- **Números grandes:** PCs en el cluster, memoria para trabajos, cores y bloques en HDFS.
- **«Ya pueden apagar» / «Todavía no apaguen las PCs»**, con el porqué. Por ejemplo: «el maestro tiene 120 de 150 bloques: HDFS está copiando lo que falta».
- **El plano:** el frente con el maestro y una fila por rack. Cada PC tiene su lámpara:
  - ámbar: sumándose;
  - verde: arriba;
  - ámbar: sin respuesta;
  - rojo: caída.

  Adentro de cada PC, sus contenedores de YARN como cuadraditos, uno por app; el del borde marcado es el ApplicationMaster.
- **Detalle por PC:** IP del túnel, último latido, bloques, memoria y cores usados.
- **Sumar PCs:** IP, código y la línea lista para copiar en Veyon.
- **La red del aula:** el tráfico por PC y por tipo (HDFS, shuffle, YARN, otro), contra un techo de referencia de 50 Mbit/s, y la tabla de quién más mueve.
- **Si una PC se cae:** la línea de tiempo de cada caída, con los contenedores perdidos, los bloques re-copiados y cuánto tardó en recuperarse.
- **Editar el plano**, **Pantalla completa** y un **modo demo** con las 11 PCs de la prueba del 27/10.
- **Apagar y prender una PC** desde su detalle, sin ir hasta la PC: **Ordenado** (avisa a YARN y a HDFS: las tareas pasan a otras PCs enseguida) o **De golpe** (como desenchufarla: se ve en «Si una PC se cae»). La PC queda con el ícono de apagada y **Prender pcNN** la vuelve a sumar. Ver «El agente de apagado», abajo.

- **Colas por grupo:** se escriben los grupos separados por coma (`grupo1, grupo2, grupo3`) y **Armar colas** crea una cola de YARN por grupo con partes iguales del aula; lo que sobra queda en `default` . Cada cola puede crecer hasta el 100 % si las otras no usan lo suyo. La barra de cada cola marca su parte, lo que usa y hasta dónde puede crecer; **Mandar un π** corre un MapReduce de muestra en esa cola. Con el campo vacío todo vuelve a `default` .
- **Dónde corrió cada tarea:** para un trabajo MapReduce o Spark, cada tarea con su localidad (data-local, rack-local u off-rack) y una barra con el resumen. Al pasar el mouse (o el foco con Tab) por una tarea, el plano marca con borde punteado las PCs que tenían el bloque y con borde lleno la PC donde corrió.
- **¿Cuánto más rápido con más PCs?:** corre siempre el mismo trabajo (π con MapReduce, π con Spark o contar palabras) con las PCs que hay y anota cuánto tardó. Con varias medidas (por ejemplo con 1, 5, 10 y 20 PCs) arma la curva de tiempo y la aceleración contra la medida con menos PCs, con la línea ideal punteada. Se exporta a CSV y se borra una medida o todas.

Se refresca cada 2 s y guarda los últimos 10 minutos de tráfico, en 300 muestras. El tráfico sale de `wg show wg0 dump` y de los contadores de nftables por par de PCs y por puerto:

Tipo de tráfico	Puertos
HDFS	9866, 9867
shuffle	13562
YARN	8030–8033, 8040–8042, 10020

El agente de apagado (TCP 8098, en cada PC)

Cada PC trabajador lanza al arrancar un agente chico (`empresa/aula/agente.py` , sólo biblioteca estándar de Python). Cuando el profe toca «Apagar», el maestro le pide al agente de esa PC que pare el DataNode y el NodeManager:

Modo	Qué hace	Qué se ve
Ordenado	`yarn --daemon stop nodemanager` y para el DataNode con sus comandos	YARN la ve SHUTDOWN enseguida; HDFS igual espera los 40 s (un DataNode no avisa que se va)
De golpe	`kill -9` al DataNode, al NodeManager y a todo Java de la PC	HDFS la da por muerta a los 40 s y YARN la pierde (LOST) al minuto

Después de apagar, el agente mira si quedó algo vivo: si el DataNode o el NodeManager siguen corriendo contesta error y la vista no la marca apagada. El contenedor y el túnel siguen andando, por eso «Prender» la vuelve a sumar.

El agente sólo atiende al maestro: con `RED=wireguard` escucha sólo en la IP del túnel de la PC y acepta pedidos de `10.13.0.1` ; con `RED=docker` , de la IP del contenedor `maestro` . Además pide el **código de la clase**. El puerto no se publica fuera del contenedor.

API del maestro: `POST /api/aula/pcs/{nodo}/apagar` (con `modo` : `ordenado` o `golpe`) y `POST /api/aula/pcs/{nodo}/prender` . Las de los otros opcionales: `GET` / `PUT /api/aula/colas` , `POST /api/aula/colas/{cola}/trabajo` , `GET /api/aula/localidad/{app}` y `GET` / `POST` / `DELETE /api/aula/escala` .

Los tiempos de una caída

Momento	Qué pasa
~10 s sin latidos	La lámpara pasa a ámbar (sin respuesta)
40 s	El NameNode da la PC por muerta y empieza a copiar sus bloques en otras
60 s	YARN la marca LOST y reintenta sus tareas en otra PC

Los dos `.cmd`

`empresa-profe.cmd` se corre en la PC del profe:

Uso	Qué hace
`empresa-profe.cmd`	Carga la imagen (del `.tar.gz` de al lado o de Docker Hub) y crea `bigdata-maestro` con el volumen `bigdata_aula` y los datos en `%USERPROFILE%\bigdata-aula`. Muestra la IP, el código y la línea de Veyon, espera «Cluster LISTO» y abre `http://localhost:8080/#aula`. Si ya estaba creado, lo prende o lo muestra
`cargar RUTA [HDFS]`	Sube un archivo o una carpeta a HDFS (por defecto `/aula/datos/`), mide la velocidad, lo anota en `cargas.txt` y muestra cómo quedaron repartidos los bloques
`estado`	Cómo está el cluster
`apagar`	Apaga el maestro en orden. Los datos quedan
`cero`	Borra el maestro y su HDFS y arranca de nuevo. Pide escribir `SI`
`hardware`	Sólo muestra los datos de la PC, sin Docker

El maestro publica el UDP 51820 y el TCP 8099 hacia la red del aula; el firewall de Windows pregunta una vez. Las webs (8080, 9870, 8088, 19888, 18080, 10000, 8888) quedan sólo en `127.0.0.1`. Si al lado del `.cmd` hay un `plano-aula.json`, lo copia al maestro.

`empresa-alumno.cmd` se manda a cada PC por Veyon:

Uso	Qué hace
`empresa-alumno.cmd IP CODIGO [CARPETA]`	Abre Docker Desktop si hace falta y carga la imagen del trabajador (de al lado, de `CARPETA` o de Docker Hub). Reserva un nombre `pcNN` y arranca `bigdata-trabajador` con ese hostname
`estado`	Cómo está esta PC en el cluster
`salir`	Saca la PC del cluster (apaga su contenedor)
`hardware`	Sólo muestra los datos de la PC

El de alumno no publica puertos ni pide administrador: sólo sale hacia la PC del profe.

Variables de entorno

Variable	De fábrica	Qué hace
`ROL`	`solo`	`solo`, `maestro` o `trabajador`
`RED`	`docker`	`docker` (aula simulada) o `wireguard` (aula real)

Variable	De fábrica	Qué hace
`MAESTRO`	`maestro`	Nombre del maestro (con `RED=docker`)
`CODIGO`	lo genera el maestro	Código de la clase
`IP_AULA`	—	IP de la PC del profe en la red del aula
`PC_NOMBRE`	—	Nombre de Windows de la PC (para el detalle)
`CARPETA_VEYON`	—	Carpeta donde Veyon deja los archivos en las PCs (arma la línea)
`LINEA_VEYON`	—	La línea completa, si se quiere fijar a mano
`PCS_POR_FILA`	`5`	PCs por fila (rack) si no hay plano
`TRABAJADOR_RAM`	con túnel, 2/3 de la RAM de Docker redondeado al GB (ej.: 3863 → 2048, 7828 → 5120); con `RED=docker`, 2048	MB para YARN en cada PC (1024–65536)
`TRABAJADOR_CORES`	con túnel, los cores que ve Docker	Cores para YARN (1–256)
`MAESTRO_NM`	`no`	Con `si`, el maestro también corre tareas
`REPLICATION`	2 con wireguard, 3 con docker	Copias de cada bloque
`WG_MTU`	`1380`	MTU del túnel
`INTERFAZ`	`si`	Con `no`, el maestro no levanta la interfaz web
`ESPERA_MAESTRO`	`600`	Segundos que una PC espera al maestro
`TERMINAL`	`0`	La terminal web viene apagada; `empresa-profe.cmd` la prende con `TERMINAL=1` (sólo localhost)
`SIMULAR_SIN_KERNEL`	—	Con `1`, fuerza `wireguard-go` (para probar el respaldo)
`AGENTE`	`si`	En una PC trabajador, con `no` no arranca el agente de apagado
`P_AGENTE`	`8098`	Puerto del agente de apagado

Qué NO hace

- **No toca la red de la facultad.** No configura routers, ni VLANs, ni DHCP. Todo pasa por un túnel entre contenedores, y la única apertura es el UDP 51820 y el TCP 8099 en la PC del profe.
- **No instala nada fijo en las PCs.** Sólo Docker Desktop, que tiene que estar instalado. El contenedor y su volumen se borran con `salir` o en la clase siguiente.
- **No es para internet.** El código de 4 letras evita que se sume una PC de otra aula por error; no es una contraseña fuerte. El túnel es privado, pero la inscripción no tiene autenticación más allá del código.
- **No mueve los cómputos al maestro.** Si no hay PCs sumadas, no hay dónde correr tareas, salvo `MAESTRO_NM=si`.

- **No guarda la clase en las PCs.** Si una PC se une a otra clase, borra su HDFS viejo. Lo que importa queda en el maestro.
- **No reemplaza a Veyon.** Lo usa para mandar los archivos y la línea; no trae un repartidor propio.
- **«Apagar» no apaga la PC.** Para el DataNode y el NodeManager; Windows y Docker siguen prendidos. No hace Wake-on-LAN, no apaga el maestro y no anda con imágenes de trabajador anteriores, que no traen el agente (la vista dice que no responde).
- **Las colas no son por alumno.** No asignan alumnos a colas ni controlan quién manda a cuál (todos los trabajos del aula los manda `root`), ni ponen prioridades o límites distintos por grupo. Hasta 12 grupos, con letras, números y `_` (hasta 20), sin `default` ni `root`. Una cola que se quita queda parada: lo que ya corría termina.
- **La localidad es de MapReduce y Spark.** Spark no informa dónde estaba el bloque (sólo dónde corrió y la localidad que logró); Tez (Hive) no se muestra, y un MapReduce recién lanzado puede no tener historial todavía.
- **La medida de escala no aísla el aula.** Mide una por vez; si cambian las PCs mientras mide, la medida queda marcada pero entra igual en la curva, y otros trabajos corriendo la ensucian. Guarda hasta 100 medidas en el maestro.

Limitaciones conocidas

- **Windows + Docker Desktop** en todas las PCs. Los `.cmd` son para Windows. En Linux o Mac se usa `docker run` a mano (ver la guía).
- **El kernel de Docker Desktop** puede no traer WireGuard: ahí entra `wireguard-go`, que necesita `/dev/net/tun`. Si tampoco está, el túnel no levanta.
- **La red del aula es el cuello.** Todo el tráfico pasa por la PC del profe (hub). El techo de 50 Mbit/s de la vista es una referencia para dibujar, no una medición del aula.
- **Probado en CI, no con PCs reales:** el agente de apagado por el túnel, lo que ve YARN al apagar (SHUTDOWN o LOST), el reparto real entre colas, el historial de un trabajo real para la localidad y la curva de escala con 20 PCs se prueban por primera vez en la prueba piloto.
- **Los racks se leen al registrarse** cada DataNode. Si cambiás el plano con la clase andando, las PCs ya sumadas siguen en su rack viejo hasta que se reinician.
- **Estado de publicación (11/11/2026):**
  - El release `aula-v2` (prerelease) tiene las tres imágenes en `.tar.gz`, los dos `.cmd` y el `LEEME.md`.
  - `:empresa` y `:empresa-trabajador` están en Docker Hub para amd64 y arm64, con la vista Aula en vivo y los opcionales.
  - Todo eso pasó las pruebas del workflow *Empresa* en CI (aula simulada y con el túnel). Con PCs reales del aula: sin verificar.

Dónde está cada cosa en el repo

Ruta	Qué hay
`Dockerfile` (etapas `wireguard-go`, `empresa`, `hadoop-trabajador`, `trabajador`)	Cómo se arman las dos imágenes
`empresa/arranque.sh`	Punto de entrada: según `ROL` y `RED`

Ruta	Qué hay
`empresa/aula/red.sh`	El túnel: `hub`, `unirse`, `reservar`
`empresa/aula/inscripcion.py`	La inscripción en el TCP 8099
`empresa/aula/topologia`	Racks
`empresa/aula/bloques.sh`	Cómo quedaron repartidos los bloques
`empresa/aula/agente.py`	El agente de apagado de cada PC (TCP 8098)
`empresa/empresa-profe.cmd`, `empresa/empresa-alumno.cmd`	Los scripts de Windows
`empresa/aula-simulada.yml`	El aula simulada (maestro + 2 PCs)
`server/sandbox_server/aula.py`, `aula_vivo.py`, `aula_yarn.py`	La API `/api/aula*` (`aula_yarn.py`: colas, localidad y escala)
`ui/src/pages/aula/`	La vista Aula
`pruebas/aula.sh`	Las pruebas del aula
`.github/workflows/empresa.yml`	Armado, pruebas, publicación y release